1 Conversion to Clause Form

Consider the sentence:**All Romans who know Marcus either hate Caesar or think that anyone who hates anyone is crazy**. It can be represented as the following wff:

$$\forall x : [Roman(x) \wedge know(x, Marcus)] \rightarrow [hate(x, Caesar) \vee (\forall y : \exists z : hate(y, z) \rightarrow thinkcrazy(x, y))]$$

The formula would be easier to work with if it is represented in ***Conjunctive normal form***. For example, the formula given above for the feelings of Romans who know Marcus would be represented in conjunctive normal form as:

$$\neg Roman(x) \vee \neg know(x, Marcus) \vee hate(x, Caesar) \vee \neg hate(y, z) \vee thinkcrazy(x, z)$$

Algorithm: Convert to Clause Form

1. **Eliminate** $\rightarrow$, using the fact that $a \rightarrow b$ is equivalent to $\neg a \vee b$. Performing this transformation on the wff given above yields:

   $$\forall x : \neg[Roman(x) \wedge know(x, Marcus)] \vee [hate(x, Caesar) \vee (\forall y : \neg(\exists z : hate(y, z)) \vee thinkcrazy(x, y))]$$

2. **Reduce the scope of each** $\neg$ to a single term, using the fact that $\neg(\neg p) = p$, deMorgan's laws [which say that $\neg(a \wedge b) = \neg a \vee \neg b$ and $\neg(a \vee b) = \neg a \wedge \neg b$], and the standard correspondences between quantifiers [$\neg\forall x : P(x) = \exists x : \neg P(x)$ and $\neg\exists x : P(x) = \forall x : \neg P(x)$]. Performing this transformation on the wff from step 1 yields:

   $$\forall x : [\neg Roman(x) \vee \neg know(x, Marcus)] \vee [hate(x, Caesar) \vee (\forall y : \forall z : \neg hate(y, z) \vee thinkcrazy(x, y))]$$

3. **Standardize variables** so that each quantifier binds a unique variable. Since variables are just dummy names, this process cannot affect the truth value of the wff. For example, the formula:

   $$\forall x : P(x) \vee \forall x : Q(x)$$

   would be converted to:

   $$\forall x : P(x) \vee \forall y : Q(y)$$

   This step is in preparation for the next.

4. **Move all quantifiers** to the left of the formula without changing their relative order. This is possible since there is no conflict among variable names. Performing this operation on the formula from step 2, we get:

   $$\forall x : \forall y : \forall z : [\neg Roman(x) \vee \neg know(x, Marcus)] \vee [hate(x, Caesar) \vee (\neg hate(y, z) \vee thinkcrazy(x, y))]$$

   At this point, the formula is in what is known as **prenex normal form**. It consists of a *prefix* of quantifiers followed by a *matrix*, which is quantifier-free.

5. **Eliminate Existential Quantifiers**

   A formula that contains an existentially quantified variable asserts that there is a value that can be substituted for the variable that makes the formula true. We can eliminate the quantifier by substituting for the variable a reference to a function that produces the desired value.

   Since we do not necessarily know how to produce the value, we must create a new function name for every such replacement. We make no assertions about these functions except that they must exist. So, for example, the formula:

   $$\exists y : President(y)$$

   can be transformed into the formula:

   $$President(S1)$$

   where $S1$ is a function with no arguments that somehow produces a value that satisfies *President*. If existential quantifiers occur within the scope of universal quantifiers, then the value that satisfies the predicate may depend on the values of the universally quantified variables. For example, in the formula:

   $$\forall x \, \exists y \; father\text{-}of(y, x)$$

   the value of y that satisfies $father\text{-}of$ depends on the particular value of x. Thus we must generate functions with the same number of arguments as the number of universal quantifiers in whose scope the expression occurs. So this example would be transformed into:

   $$\forall x \; father\text{-}of(S2(x), x)$$

   These generated functions are called *Skolem functions*. Sometimes ones with no arguments are called *Skolem constants*.

6. **Drop the prefix** At this point, all remaining variables are universally quantified, so the prefix can just be dropped and any proof procedure we use can simply assume that any variable it sees is universally quantified. Now the formula produced in step 4 appears as:

   $$[\neg Roman(x) \vee \neg know(x, Marcus)] \vee$$

   $$[hate(x, Caesar) \vee (\neg hate(y, z) \vee thinkcrazy(x, y))]$$

7. **Convert the matrix into a conjunction of disjuncts**

   In the case of our example, since there are no and's, it is only necessary to exploit the associative property of or [i.e., $(a \wedge b) \vee c = (a \vee c) \wedge (b \vee c)$] and simply remove the parentheses, giving

   $$\neg Roman(x) \vee \neg know(x, Marcus) \vee$$

   $$hate(x, Caesar) \vee \neg hate(y, z) \vee thinkcrazy(x, y)$$

However, it is also frequently necessary to exploit the distributive property [i.e., $(a \wedge b) \vee c = (a \vee c) \wedge (b \vee c)$]. For example, the formula

$$(winter \wedge wearingboots) \vee (summer \wedge wearingsandals)$$

becomes, after one application of the rule

$$[winter \vee (summer \wedge wearingsandals)]$$

$$\wedge[wearingboots \vee (summer \wedge wearingsandals)]$$

and then, after a second application, required since there are still conjuncts joined by OR's,

$$(winter \vee summer) \wedge$$

$$(winter \vee wearingsandals) \wedge$$

$$(wearingboots \vee summer) \wedge$$

$$(wearingboots \vee wearingsandals)$$

8. **Create a separate clause corresponding to each conjunct**

   In order for a wff to be true, all the clauses that are generated from it must be true. If we are going to be working with several wff's, all the clauses generated by each of them can now be combined to represent the same set of facts as were represented by the original wff's.

9. **Standardize apart the variables in the set of clauses generated in step 8**

   By this we mean rename the variables so that no two clauses make reference to the same variable. In making this transformation, we rely on the fact that

   $$(\forall x : P(x) \wedge Q(x)) = \forall x : P(x) \wedge \forall x : Q(x)$$

   Thus since each clause is a separate conjunct and since all the variables are universally quantified, there need be no relationship between the variables of two clauses, even if they were generated from the same wff.

   Performing this final step of standardization is important because during the resolution procedure it is sometimes necessary to instantiate a universally quantified variable (i.e., substitute for it a particular value). But, in general, we want to keep clauses in their most general form as long as possible. So when a variable is instantiated, we want to know the minimum number of substitutions that must be made to preserve the truth.